ADVANCED SQL QUERY SYSTEM

AIM:

To implement subqueries and nested queries in the EcoCart E-Commerce Database to analyze customer purchases, product pricing, and sales performance.

OBJECTIVES:

- To understand and implement single-row subqueries.
- To implement multi-row subqueries and nested queries.
- To identify products whose prices are above the average price.
- To analyze customers with maximum purchases.
- To identify high-value customers and top-selling products.
- To generate advanced business reports for decision-making.

IMPLEMENTATION:

1. Implement Subqueries and Nested Queries:

A subquery is a query written inside another SQL query. It helps solve complex problems by using the result of one query in another query.

The following types of subqueries are implemented:

- Single-row Subquery
- Multi-row Subquery
- Nested Subquery

```
SQL> SELECT product_name, price FROM product WHERE price > (SELECT AVG(price) FROM product);

PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE
----------
CHICKEN FRIED RICE
       140

BROWNIE
       200

CHICKEN BURGER
       200
```

2. Find Products Above Average Price:

This query is used to identify premium products in the EcoCart database.

The following details are retrieved:

- Products priced above the average product price.

```
SQL> SELECT product_name, price FROM product WHERE price = (SELECT MAX(price) FROM product);

PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE
----------
BROWNIE
       200

CHICKEN BURGER
       200
```

- Product name, category, and price.

```
SQL> SELECT p.product_name, c.category_name, p.price FROM product p JOIN category c ON p.category_id = c.category_id WHERE p.price > (SELECT AVG(price) FROM
 product);

PRODUCT_NAME
--------------------------------------------------------------------------------
CATEGORY_NAME
--------------------------------------------------------------------------------
     PRICE
----------
CHICKEN FRIED RICE
FAST FOOD
       140

BROWNIE
DESSERTS
       200

PRODUCT_NAME
--------------------------------------------------------------------------------
CATEGORY_NAME
--------------------------------------------------------------------------------
     PRICE
----------

CHICKEN BURGER
FAST FOOD
       200
```

- Expensive products in each category.

```
SQL> SELECT p.product_name, p.category_id, p.price FROM product p WHERE p.price = (SELECT MAX(p2.price) FROM product p2 WHERE p2.category_id = p.category_id
);

PRODUCT_NAME
--------------------------------------------------------------------------------
CATEGOR      PRICE
------- ----------
IDLI
C01            40

BUTTER NAAN
C02            70

BROWNIE
C05           200


PRODUCT_NAME
--------------------------------------------------------------------------------
CATEGOR      PRICE
------- ----------
CHICKEN BURGER
C03           200
```

- Products with prices higher than a selected category average.

```
SQL> SELECT p.product_name, c.category_name, p.price FROM product p JOIN category c ON p.category_id = c.category_id WHERE p.price > (SELECT AVG(p2.price) F
ROM product p2 WHERE p2.category_id = p.category_id);

PRODUCT_NAME
--------------------------------------------------------------------------------
CATEGORY_NAME
--------------------------------------------------------------------------------
    PRICE
---------
CHICKEN BURGER
FAST FOOD
      200
```

3. Identify Customers with Maximum Purchases:

Customer purchasing behavior is analyzed using SQL queries.

The following details are identified:

- Customer who spent the highest amount.

```
SQL> SELECT c.customer_id, c.customer_name, SUM(o.total_amount) AS total_spent FROM customer c JOIN orders o ON c.customer_id = o.customer_id GROUP BY c.cus
tomer_id, c.customer_name HAVING SUM(o.total_amount) = (SELECT MAX(total_sales) FROM (SELECT SUM(total_amount) AS total_sales FROM orders GROUP BY customer_
id));

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_SPENT
-----------
       102
VIJAY KUMAR
      1250
```

- Customers with the maximum number of order.

```
SQL> SELECT c.customer_id, c.customer_name, COUNT(o.order_id) AS total_orders FROM customer c JOIN orders o ON c.customer_id = o.customer_id GROUP BY c.cust
omer_id, c.customer_name HAVING COUNT(o.order_id) = (SELECT MAX(order_count) FROM (SELECT COUNT(*) AS order_count FROM orders GROUP BY customer_id));

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_ORDERS
------------
       105
KAMESH
           1

       102
VIJAY KUMAR
           1

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_ORDERS
------------

       104
SURESH
           1

       103
ANITHA
```

- Customers whose spending is above average.

```
SQL> SELECT c.customer_id, c.customer_name, SUM(o.total_amount) AS total_spent FROM customer c JOIN orders o ON c.customer_id = o.customer_id GROUP BY c.cus
tomer_id, c.customer_name HAVING SUM(o.total_amount) > (SELECT AVG(customer_total) FROM (SELECT SUM(total_amount) AS customer_total FROM orders GROUP BY cus
tomer_id));

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_SPENT
-----------
       105
KAMESH
       900

       102
VIJAY KUMAR
      1250

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_SPENT
-----------

       104
SURESH
       890
```

- Top 5 valuable customers.

```
SQL> SELECT * FROM (SELECT c.customer_id, c.customer_name, SUM(o.total_amount) AS total_spent FROM customer c JOIN orders o ON c.customer_id = o.customer_id
 GROUP BY c.customer_id, c.customer_name ORDER BY total_spent DESC) WHERE ROWNUM <= 5;

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_SPENT
-----------
       102
VIJAY KUMAR
      1250

       105
KAMESH
       900

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_SPENT
-----------

       104
SURESH
       890

       101
SNEHA

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_SPENT
-----------
       550

       103
ANITHA
       480
```

4. Generate Complex Business Queries:

Advanced SQL queries are created for business analysis.

Query 1: Best-Selling Product

Identifies products with maximum sales quantity and displays product name, total quantity sold, and total revenue generated.

```
SQL> SELECT p.product_name, SUM(od.quantity) AS total_quantity_sold, SUM(od.quantity * od.price) AS total_revenue FROM product p JOIN order_details od ON p.
product_id = od.product_id GROUP BY p.product_id, p.product_name HAVING SUM(od.quantity) = (SELECT MAX(total_quantity) FROM (SELECT SUM(quantity) AS total_q
uantity FROM order_details GROUP BY product_id));

PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD TOTAL_REVENUE
------------------- -------------
CHICKEN FRIED RICE
                  5          1250
```

Query 2: High-Value Customers

Identifies customers whose total purchase amount is greater than the average customer spending.

```
SQL> SELECT c.customer_id, c.customer_name, SUM(o.total_amount) AS total_spending FROM customer c JOIN orders o ON c.customer_id = o.customer_id GROUP BY c.
customer_id, c.customer_name HAVING SUM(o.total_amount) > (SELECT AVG(customer_spending) FROM (SELECT SUM(total_amount) AS customer_spending FROM orders GRO
UP BY customer_id));

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_SPENDING
--------------
         105
KAMESH
           900

         102
VIJAY KUMAR
          1250

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_SPENDING
--------------

         104
SURESH
           890
```

Query 3: Category Performance Analysis

Analyzes category-wise sales performance by identifying the highest revenue-generating category, number of products sold in each category, and average product price by category.

```
SQL> SELECT * FROM (SELECT c.category_name, NVL(SUM(od.quantity * od.price),
 0) AS total_revenue FROM category c LEFT JOIN product p ON c.category_id =
p.category_id LEFT JOIN order_details od ON p.product_id = od.product_id GRO
UP BY c.category_id, c.category_name ORDER BY total_revenue DESC) WHERE ROWN
UM = 1;

CATEGORY_NAME
--------------------------------------------------------------------------------
----
TOTAL_REVENUE
-------------
FAST FOOD
          2850
```

```
SQL> SELECT c.category_name, NVL(SUM(od.quantity), 0) AS products_sold FROM
category c LEFT JOIN product p ON c.category_id = p.category_id LEFT JOIN or
der_details od ON p.product_id = od.product_id GROUP BY c.category_id, c.cat
egory_name;

CATEGORY_NAME
--------------------------------------------------------------------------------
PRODUCTS_SOLD
-------------
SOUTH INDIAN
            2

FAST FOOD
            9

Beverages
            0


CATEGORY_NAME
--------------------------------------------------------------------------------
PRODUCTS_SOLD
-------------
NORTH INDIAN
            3

DESSERTS
            2
```

```
SQL> SELECT c.category_name, AVG(p.price) AS average_product_price FROM cate
gory c JOIN product p ON c.category_id = p.category_id GROUP BY c.category_i
d, c.category_name;

CATEGORY_NAME
--------------------------------------------------------------------------------
AVERAGE_PRODUCT_PRICE
---------------------
SOUTH INDIAN
                   40

FAST FOOD
                  170

NORTH INDIAN
                   70


CATEGORY_NAME
--------------------------------------------------------------------------------
AVERAGE_PRODUCT_PRICE
---------------------
DESSERTS
                  200
```

5. ADVANCED SQL REPORTS:

Report 1: Premium Product Report

This report displays product name, category, price, and stock availability condition for products whose prices are above the average product price.

```
SQL> SELECT p.product_name, c.category_name, p.price, CASE WHEN p.stock_quan
tity > 0 THEN 'Available' ELSE 'Out of Stock' END AS stock_availability FROM
 product p JOIN category c ON p.category_id = c.category_id WHERE p.price >
(SELECT AVG(price) FROM product);

PRODUCT_NAME
--------------------------------------------------------------------------
----
CATEGORY_NAME
--------------------------------------------------------------------------
----
     PRICE STOCK_AVAILA
---------- ------------
CHICKEN FRIED RICE
FAST FOOD
       140 Available

BROWNIE
DESSERTS
       200 Available

PRODUCT_NAME
--------------------------------------------------------------------------
----
CATEGORY_NAME
--------------------------------------------------------------------------
----
     PRICE STOCK_AVAILA
---------- ------------

CHICKEN BURGER
FAST FOOD
       200 Available
```

Report 2: Customer Value Report

This report displays customer name, total orders, total spending, and customer category.

```
SQL> SELECT c.customer_name, COUNT(o.order_id) AS total_orders, NVL(SUM(o.to
tal_amount), 0) AS total_spending, CASE WHEN NVL(SUM(o.total_amount), 0) >=
50000 THEN 'Premium' ELSE 'Regular' END AS customer_category FROM customer c
 LEFT JOIN orders o ON c.customer_id = o.customer_id GROUP BY c.customer_id,
 c.customer_name;

CUSTOMER_NAME
--------------------------------------------------------------------------
----
TOTAL_ORDERS TOTAL_SPENDING CUSTOME
------------ -------------- -------
KAMESH
           1            900 Regular

VIJAY KUMAR
           1           1250 Regular

SURESH
           1            890 Regular


CUSTOMER_NAME
--------------------------------------------------------------------------
----
TOTAL_ORDERS TOTAL_SPENDING CUSTOME
------------ -------------- -------
ANITHA
           1            480 Regular

SNEHA
           1            550 Regular
```

Report 3: Sales Performance Report

This report displays product-wise sales, category-wise revenue, and top-selling products.

```
SQL> SELECT p.product_name, NVL(SUM(od.quantity), 0) AS quantity_sold, NVL(S
UM(od.quantity * od.price), 0) AS total_revenue FROM product p LEFT JOIN ord
er_details od ON p.product_id = od.product_id GROUP BY p.product_id, p.produ
ct_name;

PRODUCT_NAME
--------------------------------------------------------------------------------
----
QUANTITY_SOLD TOTAL_REVENUE
------------- -------------
CHICKEN FRIED RICE
            5          1250

BUTTER NAAN
            3           720

CHICKEN BURGER
            4          1600


PRODUCT_NAME
--------------------------------------------------------------------------------
----
QUANTITY_SOLD TOTAL_REVENUE
------------- -------------
IDLI
            2           500

BROWNIE
            2           600
```

```
SQL> SELECT c.category_name, NVL(SUM(od.quantity * od.price), 0) AS total_re
venue FROM category c LEFT JOIN product p ON c.category_id = p.category_id L
EFT JOIN order_details od ON p.product_id = od.product_id GROUP BY c.categor
y_id, c.category_name;

CATEGORY_NAME
--------------------------------------------------------------------------------
----
TOTAL_REVENUE
-------------
SOUTH INDIAN
         500

FAST FOOD
        2850

Beverages
           0


CATEGORY_NAME
--------------------------------------------------------------------------------
----
TOTAL_REVENUE
-------------
NORTH INDIAN
         720

DESSERTS
         600
```

Report 4: Business Decision Report

This report identifies:

- High-performing products.

```
SQL> SELECT * FROM (SELECT p.product_name, SUM(od.quantity) AS quantity_sold
  FROM product p JOIN order_details od ON p.product_id = od.product_id GROUP
BY p.product_id, p.product_name ORDER BY quantity_sold DESC) WHERE ROWNUM <=
 5;

PRODUCT_NAME
--------------------------------------------------------------------------------
----
QUANTITY_SOLD
-------------
CHICKEN FRIED RICE
            5

CHICKEN BURGER
            4

BUTTER NAAN
            3


PRODUCT_NAME
--------------------------------------------------------------------------------
----
QUANTITY_SOLD
-------------
IDLI
            2

BROWNIE
            2
```

- Low-performing products.

```
SQL> SELECT p.product_name, SUM(od.quantity) AS quantity_sold, SUM(od.quanti
ty * od.price) AS total_revenue FROM product p JOIN order_details od ON p.pr
oduct_id = od.product_id GROUP BY p.product_id, p.product_name HAVING SUM(od
.quantity) >= (SELECT AVG(product_quantity) FROM (SELECT SUM(quantity) AS pr
oduct_quantity FROM order_details GROUP BY product_id)) ORDER BY total_reven
ue DESC;

PRODUCT_NAME
--------------------------------------------------------------------------------
QUANTITY_SOLD TOTAL_REVENUE
------------- -------------
CHICKEN BURGER
            4          1600

CHICKEN FRIED RICE
            5          1250
```

- High-value customers.

```
SQL> SELECT c.customer_name, SUM(o.total_amount) AS total_spending FROM cust
omer c JOIN orders o ON c.customer_id = o.customer_id GROUP BY c.customer_id
, c.customer_name HAVING SUM(o.total_amount) > (SELECT AVG(customer_spending
) FROM (SELECT SUM(total_amount) AS customer_spending FROM orders GROUP BY c
ustomer_id)) ORDER BY total_spending DESC;

CUSTOMER_NAME
--------------------------------------------------------------------------------
----
TOTAL_SPENDING
--------------
VIJAY KUMAR
         1250

KAMESH
          900

SURESH
          890
```

- Revenue trends.

```
SQL> SELECT TO_CHAR(order_date, 'YYYY-MM') AS sales_month, COUNT(order_id) A
S total_orders, SUM(total_amount) AS monthly_revenue FROM orders GROUP BY TO
_CHAR(order_date, 'YYYY-MM') ORDER BY sales_month;

SALES_M TOTAL_ORDERS MONTHLY_REVENUE
------- ------------ ---------------
2026-03            1             480
2026-05            2            2140
2026-09            1             550
2026-10            1             900
```